

Yellowstone Database: Moving to a Nonrelational database

Marcela Isabel Redondo Hernandez

Data Architecture for Business Analytics Project

2025

Yellowstone Database: Moving to a Nonrelational database

The Yellowstone database is currently structured using a relational model that organizes data into normalized tables such as `park_locations`, `visitors`, `monthly_visitors`, `weather`, `wildlife`, `observation_period`, and `wildlife_weather_impact`. Each table captures a specific domain of park operations: geographic regions and attractions, yearly and monthly visitor counts, recorded climate variables, species population data, seasonal observation patterns, and the influence of weather on species visibility. The `monthly_visitors` table is tied to a trigger that dynamically recalculates annual visitor totals, improving data accuracy and minimizing manual updates. The `wildlife_weather_impact` table continues to serve as a junction point, correlating species behavior with environmental conditions at specific park locations. While this normalized design ensures data integrity and supports detailed analysis, it also requires frequent joins across multiple tables to answer complex queries. As the park increasingly collects semi-structured data (such as ranger notes, GPS-based animal sightings, and real-time sensor feeds) the limitations of a strictly relational approach become more evident. These developments highlight the potential advantages of adopting a more flexible, schema-less model like MongoDB to support evolving data needs.

Pros and cons of moving to a nonrelational database (MongoDB)

Pros

- **Schema flexibility for sensor and observation data**
 - **Schema:** MongoDB allows dynamic data storage, which means sensor and observation data can evolve without altering the schema.

- **Database:** This flexibility makes real-time data logging easier, accommodating new types of data or metadata without disruption.
- **Embedding improves query performance**
 - **Schema:** By embedding related data (e.g., species, weather) within documents, MongoDB eliminates the need for complex joins.
 - **Database:** This boosts query performance by reducing database operations, making data retrieval faster and more efficient.
- **GeoJSON support for park regions and sightings**
 - **Schema:** MongoDB natively supports GeoJSON data for geospatial queries.
 - **Database:** This feature enhances location-based analysis, making it easier to track sightings and park boundaries using GPS coordinates.
- **Scalable visitor logs**
 - **Schema:** MongoDB allows storing visitor logs as documents per year or location, reducing the need for large table aggregations.
 - **Database:** This setup improves scalability, making time-series analysis of visitor data faster and more efficient.
- **Richer observation metadata**
 - **Schema:** MongoDB's flexible model lets you store diverse and unexpected metadata without schema adjustments.
 - **Database:** This flexibility enables richer wildlife observation data collection, capturing more detailed field notes and behaviors.

Cons:

- **Complex migration from normalized tables**

- **Schema:** Migrating data from normalized relational tables to MongoDB's document-based model requires restructuring relationships (e.g., species_id, weather_id).
- **Database:** This transformation can be complex and requires careful data mapping, which may introduce challenges during migration.
- **Transactional limitations**
 - **Schema:** While MongoDB supports multi-document transactions, they may not be as robust or efficient as those in relational databases
 - **Database:** Complex transactional operations (e.g., syncing data across locations) may require extra planning and could lead to performance bottlenecks.
- **Data validation is application-driven**
 - **Schema:** Unlike SQL databases, MongoDB doesn't enforce data types or constraints, requiring application-level validation.
 - **Database:** This means that maintaining data integrity becomes the responsibility of the application, which can introduce errors if not carefully managed.
- **Reporting tool compatibility**
 - **Schema:** MongoDB's non-relational structure isn't directly compatible with SQL-based reporting tools like Tableau.
 - **Database:** Middleware or export processes may be necessary to analyze MongoDB data in these tools, adding extra steps to the reporting process.

Recommendation

I strongly recommend upgrading key components of the Yellowstone database to a non-relational system like MongoDB. This shift is necessary to support the park's growing reliance

on dynamic, semi-structured data such as ranger field notes, GPS-tagged wildlife sightings, and real-time sensor input. MongoDB's schema flexibility, embedded document structure, and native geospatial capabilities are well-aligned with these needs (MongoDB, n.d.; Insightsoftware, 2023).

While structured data such as park infrastructure and long-term weather patterns continue to benefit from relational storage, these elements can remain in the current PostgreSQL system. This hybrid architecture will allow us to modernize the data ecosystem without disrupting existing tools or compromising data integrity (EASA Software, 2022; DATAVERSITY, 2022).

By adopting this dual system strategy:

- We immediately improve our ability to ingest and analyze unstructured and location-based data.
- We maintain transactional reliability and reporting compatibility for structured historical datasets.
- We create a future-proof foundation that can scale as new data sources emerge.

Deployment plan for upgrading to a nonrelational database (MongoDB)

To implement the proposed upgrade, we recommend a phased deployment strategy. This plan focuses on migrating high-impact, semi-structured datasets (such as wildlife sightings, ranger notes, and sensor data) to MongoDB, while maintaining core structured data in the existing PostgreSQL system.

Phase 1: Planning and assessment

- Identify which data types and modules are best suited for MongoDB (e.g., observation notes, GPS sightings, real-time sensor feeds).
- Review data volume, access frequency, and integration points with other systems.
- Define performance goals and success metrics for the MongoDB deployment.

Phase 2: Schema design and prototyping

- Design MongoDB collections for each target dataset using a flexible, embedded schema structure.
- Leverage MongoDB's GeoJSON and time-series capabilities for geospatial and chronological data.
- Build a prototype to validate data modeling choices, querying performance, and storage efficiency.

Phase 3: Data migration and testing

- Extract, transform, and load (ETL) data from relational tables into MongoDB collections using scripts or data integration tools.
- Maintain data integrity by mapping foreign key relationships into embedded subdocuments or referenced IDs.
- Run extensive tests to ensure completeness, accuracy, and performance under expected loads.

Phase 4: Application integration and hybrid sync

- Update applications and dashboards to interact with both PostgreSQL and MongoDB where necessary.

- Implement APIs or middleware to unify data access and abstract differences between systems.
- Set up periodic synchronization jobs for modules that temporarily require data to coexist in both databases.

Phase 5: monitoring, optimization, and training

- Continuously monitor MongoDB performance, query patterns, and system health using built-in tools.
- Refine indexes and document structures based on real usage.
- Provide training for staff and developers on MongoDB best practices and data handling.

By following a phased deployment plan, Yellowstone can modernize its data infrastructure responsibly and efficiently.

References

- DATAVERSITY. (2022, January 25). NoSQL databases: Advantages and disadvantages. <https://www.dataversity.net/nosql-databases-advantages-and-disadvantages/>
- EASA Software. (2022, January 25). 8 advantages of a relational database. <https://www.easasoftware.com/insights/8-advantages-of-a-relational-database/>
- Insightsoftware. (2023, June 13). What's the difference? Relational vs. non-relational databases. <https://insightsoftware.com/blog/whats-the-difference-relational-vs-non-relational-databases/>
- MongoDB. (n.d.). Relational vs. non-relational databases. <https://www.mongodb.com/resources/compare/relational-vs-non-relational-databases>
- MongoDB. (n.d.). NoSQL explained: A guide to non-relational databases. MongoDB. <https://www.mongodb.com/es/resources/basics/databases/nosql-explained>
- Nguyen, L. (2024, December 13). NoSQL databases visually explained with examples. AltexSoft. <https://www.altexsoft.com/blog/nosql-databases/>